

Neural Radiance Cache Implementation on Mobile GPU

Collin Allen*

Samsung Advanced Computing Laboratory
San Jose, USA
collin.allen@samsung.com

Luca Quartesan*

Traverse Research BV
Breda, Netherlands
Breda University of Applied Sciences
Breda, Netherlands
luca.qrtsn@gmail.com

Jasper de Winther*

Traverse Research BV
Breda, Netherlands
jasperdewinther@gmail.com

Jasper Bekkers

Traverse Research BV
Breda, Netherlands
jasper@traversereseearch.nl



Figure 1: A scene from the Evolve benchmark [Traverse Research 2025] with a large mirror in the shape of an arch on the left side of the image. We directly sample the Neural Radiance Cache at the second vertex, meaning that after we trace a new ray from the mirror, we directly visualize the cache. Shadows, specular highlights, as well as general global illumination are being represented by the cache. This capture was taken with our desktop-quality preset.

Abstract

We present a Neural Radiance Cache (NRC) integrated into the global illumination and reflection passes of a hybrid renderer targeting mobile devices. Real-time Monte Carlo rendering on mobile remains challenging due to limited compute and memory bandwidth, despite the emergence of hardware-accelerated ray tracing on mobile GPUs. Ray tracing workloads often exhibit incoherent memory access patterns that can bottleneck GPU performance. In contrast, our NRC implementation shifts this cost toward predictable, compute-centric neural inference. Our design features a fused MLP architecture implemented entirely in compute shaders, along with an efficient path-traced training pipeline decoupled from inference. While introducing more overhead than traditional radiance cache techniques, our mobile-optimized NRC achieves higher perceptual quality while maintaining real-time frame rates across a range of scenes on modern mobile hardware.

*These authors contributed equally to this work



This work is licensed under a Creative Commons Attribution 4.0 International License.
SA Technical Communications '25, Hong Kong, Hong Kong
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2136-6/25/12
<https://doi.org/10.1145/3757376.3771399>

CCS Concepts

• Computing methodologies → Neural networks; Ray tracing.

Keywords

Global Illumination, Neural Networks, Ray Tracing, Mobile Rendering

ACM Reference Format:

Collin Allen, Luca Quartesan, Jasper de Winther, and Jasper Bekkers. 2025. Neural Radiance Cache Implementation on Mobile GPU. In *SIGGRAPH Asia 2025 Technical Communications (SA Technical Communications '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3757376.3771399>

1 Introduction

In the push to achieve more photorealistic rendering, Monte Carlo algorithms have continued to grow in popularity. However, due to the stringent compute and power constraints of mobile GPUs, such algorithms have remained limited in their complexity and impact.

Approximation techniques, such as radiance caching [Krivaneck et al. 2005], can reduce the computation needed to achieve stable and high-quality results. Radiance caching consists of storing outgoing radiance values for future use, reducing the number of rays and bounces needed for radiance estimates. However, most radiance caching algorithms have significant drawbacks, particularly

for mobile rendering. Often radiance caches rely on spatial data structures with complex allocation strategies to efficiently store radiance values, which can be too memory-intensive on limited mobile hardware. Many approaches also rely on precomputation, often called light baking, which limits the ability of the cache to handle dynamic content; this is a significant drawback for many real-time rendering uses. One new development in radiance caching techniques is Neural Radiance Caching (NRC) [Müller et al. 2021], which is designed to address several of these drawbacks.

NRC applies these design principles by implicitly storing radiance within the weights of a Neural Network (NN), rather than relying on traditional data structures. The system operates through online training, where the NN continuously adapts during rendering by being optimized with samples generated in each frame. For rendering, short paths are traced and terminated into the NRC once sufficient area spread is achieved, reducing artifacts in specular reflections. To train the cache, a small fraction of these rendering paths are extended by additional vertices, with radiance estimates collected along these longer paths serving as reference values for network training. The baseline implementation [Müller et al. 2021] employs a fully fused multi-layer perceptron (MLP) in CUDA, optimized for modern NVIDIA GPUs with tensor cores and can be found in the tiny-cuda-nn library [Müller 2021].

To investigate the potential benefits of NRC for mobile hardware, we have implemented a variant of the original NRC algorithm within a hybrid renderer. Our implementation is integrated within the diffuse global illumination (GI) and reflection passes of this hybrid renderer, providing a significant improvement in image quality over traditional techniques. Finally, we optimize our implementation to achieve real-time frame rates on a mobile GPU.

2 Hybrid Renderer Integration

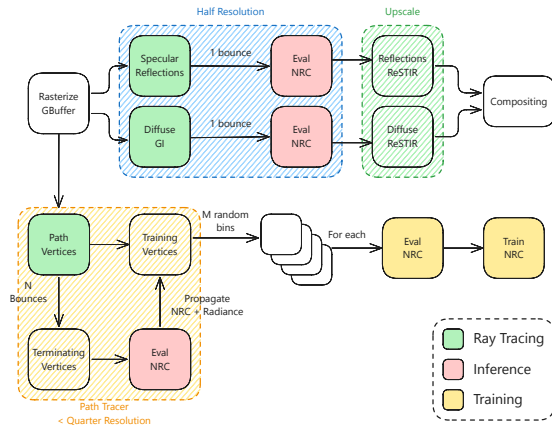


Figure 2: Overview of one frame of the hybrid rendering pipeline with NRC.

The integration of NRC within our hybrid rendering pipeline is illustrated in Fig. 2. Our hybrid renderer uses rasterization for G-buffer generation, while applying ray tracing (RT) for specific passes: direct sunlight shadows, diffuse GI and specular reflections.

Our implementation of NRC follows the core principles outlined by Müller et al. [2021] integrated in the diffuse GI and specular reflections passes of our hybrid renderer, instead of a path tracer. These passes run at reduced resolution, typically one-quarter to one-half of full screen resolution, providing essential quality performance control for achieving real-time frame rates on mobile hardware. To maximize our limited ray budget, we integrate Reservoir-based Spatiotemporal Importance Resampling (ReSTIR) into both passes, enabling spatial and temporal reuse of radiance samples without tracing additional rays, as described in previous work [Bouma 2025].

Diffuse GI is computed by tracing a single diffuse ray from each primary surface point and querying the NRC at the surface intersection (second vertex). Similarly, the reflection pass computes ray-traced specular reflections by querying the NRC at reflection ray hit points. For both passes we always query the cache at the second vertex, rather than continuing to trace additional rays for incoming radiance, which is crucial for mobile performance. The neural approximation provides visually plausible results, even with perfect mirrors, while making real-time ray-traced reflections feasible on mobile hardware.

3 Training Pipeline

To update the cache, we deploy a standalone path tracer instead of integrating data collection into our hybrid rendering pipeline, allowing them to run at different resolutions. The training phase collects radiance samples by tracing rays from a subset of primary surface points for at most N bounces, we find $N=5$ to be enough, and performing next event estimation at each path vertex except the last. At the terminal vertex, we query the NRC to approximate incoming radiance, enabling self learning that simulates infinite-bounce lighting.

Network Configuration. We experimented with networks of 32 features wide, instead of 64, and 4 layers, to better fit mobile hardware constraints. For positional encoding, we tested both triangle wave frequency encoding, as in the baseline [Müller et al. 2021], and latent grid encoding with trilinearly sampled multi-resolution hash grids [Müller et al. 2022]. Although the baseline [Müller et al. 2021] shows good results using OneBlob to encode auxiliary inputs (surface normal, view direction, roughness), we found that providing them as is provides good enough results. This helped reduce the size of the first layer of the MLP and lower bandwidth requirements.

Training Details. We partition training data into randomly shuffled fixed-size bins with one optimization step per bin. We use a relative mean squared error (RMSE) loss, where RGB ground truth and prediction are normalized by the square of the prediction. Furthermore, we compute the loss in logarithmic space by applying $\ln(x + 1)$ to the target, since the network often fails to converge without this transformation, likely due to fp16 precision. Optimization uses Adam with learning rates of 0.001 for the network and 0.1 for latent grid encoding. As suggested in the previous work [Müller et al. 2021], we keep two sets of weights, one for training and one for inference, and we use an exponential moving average to apply

the training weights to the ones used during inference, resulting in a smoother cache.

4 Network Optimization

Our mobile GPU implementation required significant adaptations from the baseline [Müller et al. 2021], due to differences in hardware. Compared to desktop, our target mobile GPU has considerably less local data share (LDS), also called group shared memory, and no hardware acceleration for NNs available from shaders.

Inference. The baseline implementation processes mini-batches of 128 items with intermediate results stored in LDS. With each layer’s output requiring data that over saturates usage of our LDS capacity, we found reducing batch size to 64 items optimal through experimentation, while further reduction to 32 items decreased overall performance despite fitting in LDS.

Since LDS is heavily utilized for intermediate outputs, we leverage the L2/L1/L0 cache hierarchy of the RDNA3.5 architecture. The network weights are relatively small (6280 bytes) and fit entirely in the L0 cache (16 KB per compute unit). Between the initial input load and output store, loading weights is the only memory operation, maintaining weights in cache until thread group execution completes. Finally, our implementation uses 256-thread groups.

Our use case runs inference four times per frame: during diffuse GI and reflection passes, on terminating training vertices for self-training, and for computing training vertex predictions.

Training Forward. Training requires computation of gradients with respect to each layer’s input $\partial\mathcal{L}/\partial x_l$ and weights $\partial\mathcal{L}/\partial\theta_l$. When computing these gradients, we need each layer’s input x_l . For all layers after the first, its input corresponds to the previous layer’s output and an activation. During the training forward pass we write each layer’s output h_l before activation to global memory, while keeping it in LDS to be processed by the next layer. We write these intermediate outputs as is to calculate $\partial\mathcal{L}/\partial x_l$, and transposed to calculate $\partial\mathcal{L}/\partial\theta_l$. Writing them out while in LDS offers a perfect opportunity to do coalesced writes for both layouts.

Backward. Since we need each layer’s input gradient to compute weight gradients $\partial\mathcal{L}/\partial\theta_l$, we cannot fuse the backward pass, and instead is split into two.

For input gradients, we run a single fused kernel computing $\partial\mathcal{L}/\partial h_l$ for each layer starting from the last. After each layer computation, since it will be used for the previous layer, we temporarily store the gradient $\partial\mathcal{L}/\partial h_l$ in LDS, like during inference. This is then written to global memory transposed, to be used in the previous layer’s weight gradient $\partial\mathcal{L}/\partial\theta_{l-1}$ calculation.

Weight gradient computation faces challenging memory access patterns since the inner dimension K (batch size) requires reduction, a dot product, over 16384 elements that are not adjacent in memory. We address this by storing both output gradients and inputs in transposed form from previous passes as mentioned above. Finally, we dispatch kernels with large thread groups (256) targeting small 2×2 tiles of elements in the gradient, approaching this operation as a reduction, rather than a matrix multiplication, optimized by utilizing LDS and wave intrinsics.

5 Analysis and Results

We implemented all features within Breda [Traverse Research 2025], a proprietary cross-platform graphics research framework developed by Traverse Research, written in Rust with support for NN operations. All NN work was implemented in compute shaders, primarily using half-precision (fp16), with few exceptions during training. Our rendering pipeline utilizes Vulkan ray tracing extensions, performs calculations in ACEScg color space, and employs Breda’s multi-layered material model [Vernooij 2024]. All shaders were written in HLSL using Shader Model 6.6 and compiled to SPIR-V through DirectX Shader Compiler to run on Android via Vulkan. Timings were collected on Samsung’s Xclipse 950 mobile GPU, an RDNA-3.5 based processor with ray tracing acceleration.

Comparison Implementations. We compare our NRC implementation against the existing radiance cache implementation in Breda [Bouma 2025], a clipmap based irradiance cache [Panteleev 2014]. Both the clipmap cache and NRC inject their radiance at the second vertex. We compare quality using the final render target of the hybrid renderer against Breda’s path tracer, using 4,096 samples per pixel with a maximum of 8 bounces per sample. As shown in Figure 3, the images compared display only indirect lighting, as this represents the contribution of the cache within the hybrid rendering pipeline. In other words, we exclude direct lighting incoming at the first vertex, isolating precisely what the radiance cache would contribute in the final frame.

Test Scenes. For generating realistic comparisons, we selected four sample scenes: Living Room, Archvis, Bistro, and Pink Room. Three of the scenes, Living Room, Archvis, and Pink Room are interior where indirect illumination has a large impact on the lighting and visual quality. The Bistro scene is an outdoor scene, adding more diversity to the lighting scenarios. We evaluated visual quality over scenes using Flip [Andersson et al. 2020] as shown in Table 1.



Figure 3: Visual comparison of the "Living Room" scene. Left figure: Indirect lighting only, comparing path tracer reference (left) with our hybrid NRC approach (right). Right figure: Comparison of the path tracer including direct lighting (left) and NRC directly visualized at the primary (right).

5.1 Performance Analysis

Inference. Our NRC implementation achieves a balance between high visual quality and performance on mobile GPUs through careful optimization of the NRC network architecture for efficient

shader utilization and memory throughput on the Xclipse 950 GPU. After searching the space of hyperparameters, we identified an optimal configuration for balancing visual quality and performance.

We found that a small MLP with four layers and 32 hidden features provided the best balance between performance and visual quality. The positional encoding scheme has a significant impact on runtime and quality. We implemented both the triangle wave frequency encoding scheme used in [Müller et al. 2021] and the latent grid encoding of [Müller et al. 2022].

On mobile GPUs, the 16-level latent grid from [Müller et al. 2022] performs poorly due to the high memory bandwidth demands of accessing features stored in the grid. For example, inference at 480×270 resolution takes approximately 20 ms on the Xclipse GPU. However, reducing the grid to just 4 levels brings the runtime down to around 3 ms, with only a modest reduction in visual quality. In contrast, frequency encoding avoids these memory operations entirely by computing features, requiring only 0.5 ms for inference at the same resolution.

The bulk of inference time is consumed by the shaders for the frequency encoding and MLP forward pass. The fused MLP executes the full network inference in 2.3 ms at 480×270 resolution on the Xclipse 950 GPU. Through extensive optimizations we achieve a VALU utilization of 72.8% on the Xclipse GPU. This efficiency is crucial on mobile. Including all supporting data processing shaders, the total inference overhead is about 4 ms per frame for both the GI and Reflections passes. Importantly, this 4 ms cost is consistent and scene-independent, allowing for aggressive shader-level optimizations that would be more difficult under variable workloads.

Training. Several parameters provide a tradeoff between performance and quality, particularly the training resolution and the number of training bins. We found that a training resolution of 128×128 (16,384 samples) provided the best tradeoff, taking roughly 2 ms to generate the training paths. Processing the entire 16,384 training elements in a single bin takes 2.2 ms for the backward pass, spending 1.5 ms on weight gradients and 0.4 ms on input gradients, and 1.2 ms for inference and forward pass during training. In total, we spend approximately 4 ms for the aforementioned passes, optimizer, and all shaders involved in training the network. We recommend using three or four smaller bins for maximal quality, though this incurs additional runtime cost, since processing one large bin is more efficient than multiple smaller bins covering the same samples.

Full Frame Time. The full frame time is approximately 31 ms with NRC, while our clipmap baseline generally achieves 30 FPS for these scenes on the same device, though at reduced quality compared to our radiance cache approach.

6 Conclusion and Future Work

We presented an integration of Neural Radiance Caching (NRC) into a hybrid renderer for mobile devices, enabling high-quality GI under tight memory and compute constraints. Our system decouples training and inference, leverages reduced-resolution ray tracing, and implements an optimized MLP algorithm for half-precision inference and training that fuses operations and minimizes memory reads and writes by leveraging LDS and low-level caches.

Table 1: Average Ψ LIP error (lower is better) per scene for each method. NRC (Latent Grid), achieves the lowest average across all 4 scenes.

Scene	Clipmap	NRC (Latent Grid)	NRC (Frequency)
Living Room	0.450	0.327	0.342
Archvis	0.575	0.412	0.455
Bistro	0.809	0.649	0.644
Pink Room	0.282	0.249	0.254
Average	0.529	0.409	0.423

Through a series of ablations and comparisons, we demonstrated that a NN-based cache achieves lower perceptual error compared to traditional radiance caches, while running at real-time frame rates. Even with compact networks, half as wide as the baseline, NRC maintains high-quality results across different scenes, highlighting its viability for future real-time rendering applications on low-power hardware.

As mobile GPUs continue to improve, we expect several of the computations in this implementation to see speedups. In particular, the ray tracing for data generation will benefit from hardware acceleration, and the network operations of backpropagation and inference will benefit from further hardware acceleration for matrix multiplications accessible through shaders, like WMMA and cooperative vectors. These improvements, combined with our efficient inference implementation, could bring this advanced technique within reach of real-time rendering for mobile games.

Finally, while our method queries the cache at secondary intersections, it would be ideal to be able to evaluate it at primaries. Although direct visualization of NRC at primary vertices presents a good approximation, the quality is not production-ready, as shown in Figures 1 and 3. Recent work introduces the Neural Incident Radiance Cache (NIRC) [Dereviannykh et al. 2024], which models incident radiance rather than outgoing radiance, and enables queries directly at primary vertices. We leave exploration of this integration for future work.

References

- Pontus Andersson, Jim Nilsson, Tomas Akenine-Möller, Magnus Oskarsson, Kalle Åström, and Mark D. Fairchild. 2020. Ψ LIP: A Difference Evaluator for Alternating Images. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 3, 2 (2020), 15:1–15:23.
- Darius Bouma. 2025. Diffuse Global Illumination. In *GPU Zen 3: Advanced Rendering Techniques*, Wolfgang Engel et al. (Eds.). Independently published, Chapter 14.
- Mikhail Dereviannykh, Dmitrii Klepikov, Johannes Hanika, and Carsten Dachsbacher. 2024. Neural Two-Level Monte Carlo Real-Time Rendering. arXiv:2412.04634 [cs.GR] <https://arxiv.org/abs/2412.04634>
- J. Krivanek, P. Gauthier, S. Pattanaik, and K. Bouatouch. 2005. Radiance caching for efficient global illumination computation. *IEEE Transactions on Visualization and Computer Graphics* 11, 5 (2005), 550–561. doi:10.1109/TVCG.2005.83
- Thomas Müller. 2021. *tiny-cuda-nn*. <https://github.com/NVlabs/tiny-cuda-nn>
- Thomas Müller, Fabrice Rousselle, Jan Novák, and Alexander Keller. 2021. Real-time neural radiance caching for path tracing. *ACM Trans. Graph.* 40, 4, Article 36 (July 2021), 16 pages. doi:10.1145/3450626.3459812
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics* 41, 4 (July 2022), 1–15. doi:10.1145/3528223.3530127
- Alexey Panteleev. 2014. Practical Real-Time Voxel-Based Global Illumination For Current GPUs. NVIDIA, GPU Technology Conference.
- Traverse Research. 2025. *Evolve Benchmark*. <https://www.evolvebenchmark.com/>
- Hannes B. Vernooij. 2024. A Fresnel Model for Coated Materials. In *Eurographics 2024 - Short Papers*, Ruizhen Hu and Panayiotis Charalambous (Eds.). The Eurographics Association. doi:10.2312/egs.20241028